Data and Artificial Intelligence

Cyber Shujaa Program

Week 11 Assignment
Natural Language Processing with Transformers

Student Name: Salome Kungu

Student ID: CS-DA03-26054

Contents

Data and Artificial Intelligence 1
Cyber Shujaa Program 1
Week 11 Assignment Natural Language Processing with Transformers 1
Introduction 2
The key objectives of the assignment were to: 2
Tasks Completed 2
Task 1: Import Libraries and Load BERT Model 2
Task 2: Define Sentence Pairs 4
Task 3: Extract Sentence Embeddings 5
Task 4: Compute Cosine Similarity and Apply Similarity Threshold 6
Task 5: Evaluate Model Accuracy 8
Answers to the Extra Assignment Questions 9
Conclusion 10

Introduction

This assignment focuses on applying Natural Language Processing (NLP) techniques using the BERT (Bidirectional Encoder Representations from Transformers) model. The goal of the project is to understand how contextual embeddings generated by BERT can be used to measure semantic similarity between sentences.

The assignment demonstrates how pre-trained transformer models from Hugging Face can encode sentences into dense vector representations. These embeddings capture contextual meaning, allowing the model to distinguish similar and unrelated sentences.

The project involves computing cosine similarity between sentence embeddings and predicting whether sentence pairs are semantically similar based on a defined threshold.

The key objectives of the assignment were to:

1. Apply a pre-trained BERT model for sentence encoding
2. Extract contextual embeddings using the [CLS] token
3. Compute cosine similarity between sentence embeddings
4. Classify sentence pairs as similar or not similar
5. Evaluate prediction accuracy using manually assigned labels

Tasks Completed

Link to Code via Google Colab:

https://colab.research.google.com/drive/1h6cZCK2xMTj8sUjkfUi6hPO60Xaf25iU?usp=sharing

Task 1: Import Libraries and Load BERT Model

What

- Imported libraries such as Transformers, TensorFlow, NumPy, and Scikit-learn.
- Loaded the bert-base-uncased tokenizer and model from Hugging Face.

Why

- The tokenizer converts text into tokens understood by the BERT model.
- The pre-trained BERT model generates contextual embeddings for sentences.

Salome_Kungu_CS-DA03-26054_NLP_with_transformers_Project.ipynb

```python
# STEP 1: Install Required Libraries
# Install compatible versions

!pip install -q transformers
!pip install -q tensorflow
!pip install -q scikit-learn
```

```python
# STEP 2: Import Libraries
# Import required libraries

from transformers import BertTokenizer, TFBertModel
import tensorflow as tf
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

print("Libraries loaded successfully!")
```

```
Libraries loaded successfully!
```

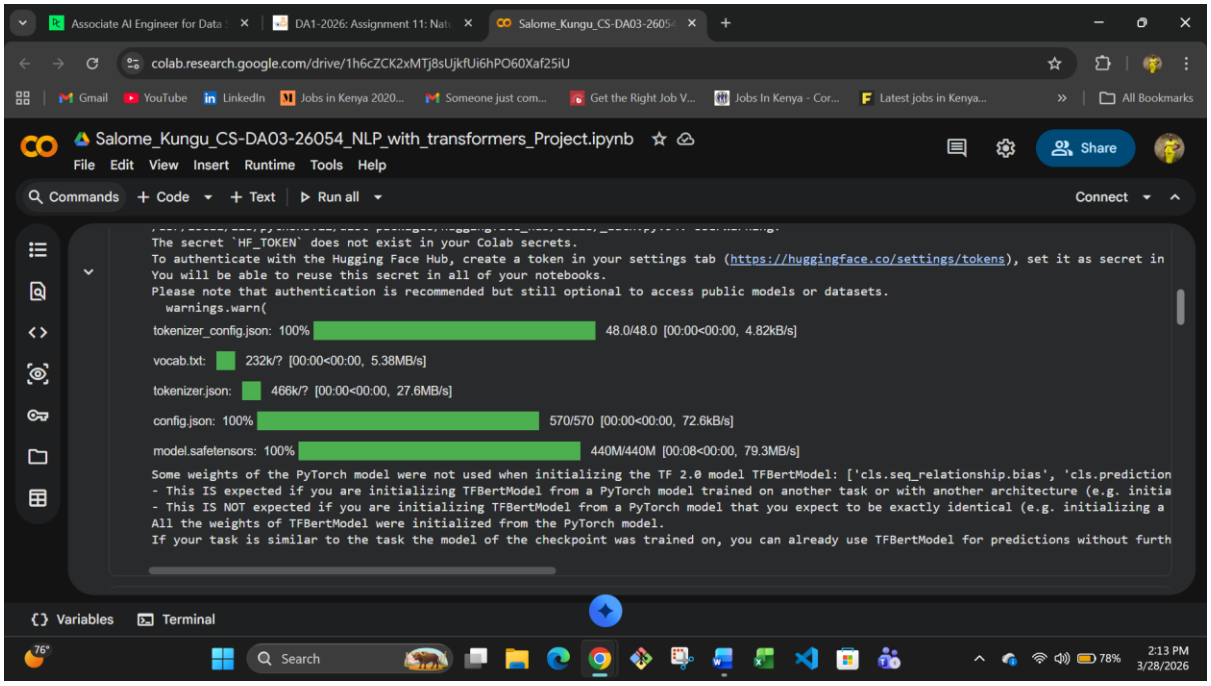
Salome_Kungu_CS-DA03-26054_NLP_with_transformers_Project.ipynb

```
Libraries loaded successfully!
```

```python
# STEP 3: Load Pretrained BERT
# Load pre-trained BERT tokenizer and BERT model from Hugging Face

tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
bert_model = TFBertModel.from_pretrained('bert-base-uncased')
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/file_download.py:949: FutureWarning: `resume_download` is deprecated and will be remo
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret in
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%  48.0/48.0 [00:00<00:00, 4.82kB/s]
vocab.txt: 232k/? [00:00<00:00, 5.38MB/s]
```



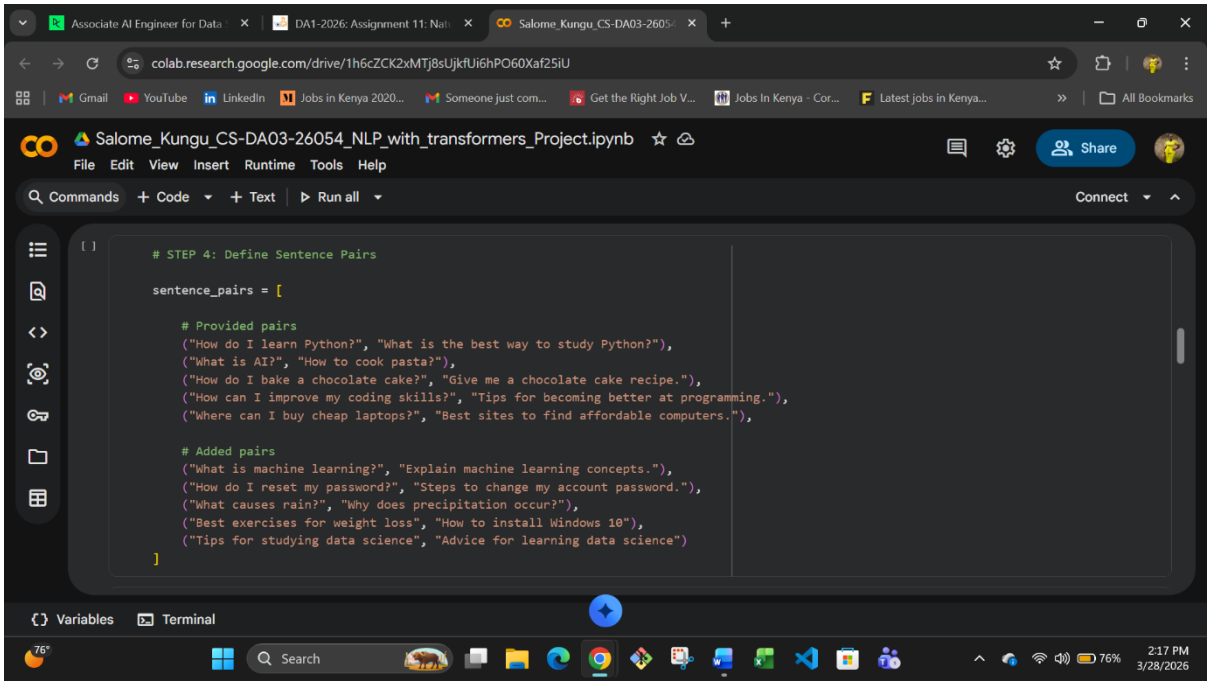
Task 2: Define Sentence Pairs

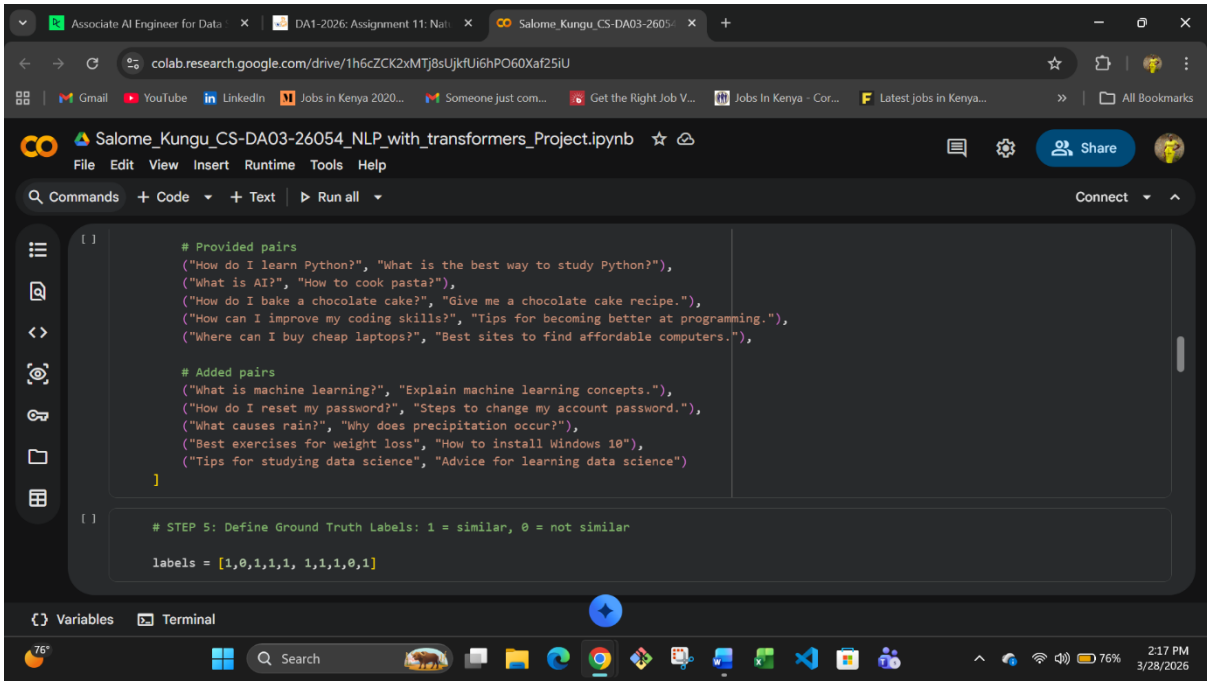
What

- Created 10 sentence pairs including 5 provided examples and 5 additional custom examples.

Why

- These pairs were used to evaluate semantic similarity between sentences.





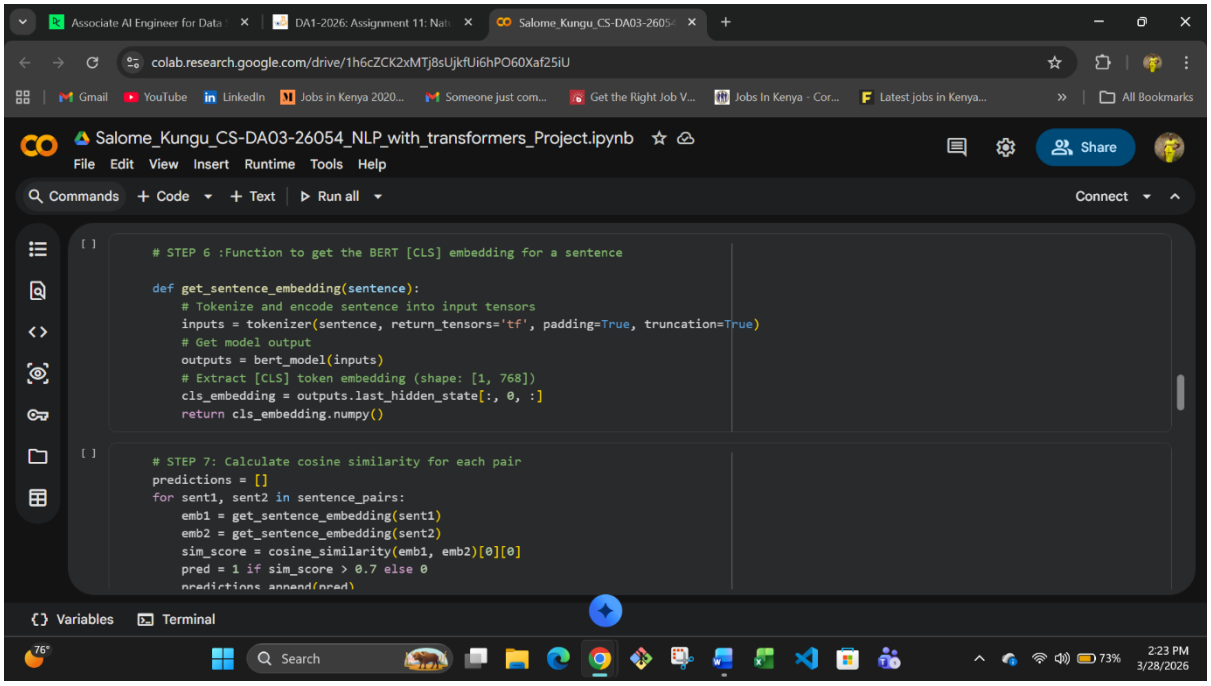
Task 3: Extract Sentence Embeddings

What

- Used the [CLS] token representation from the BERT model output.

Why

- The [CLS] token captures the overall semantic meaning of the sentence.



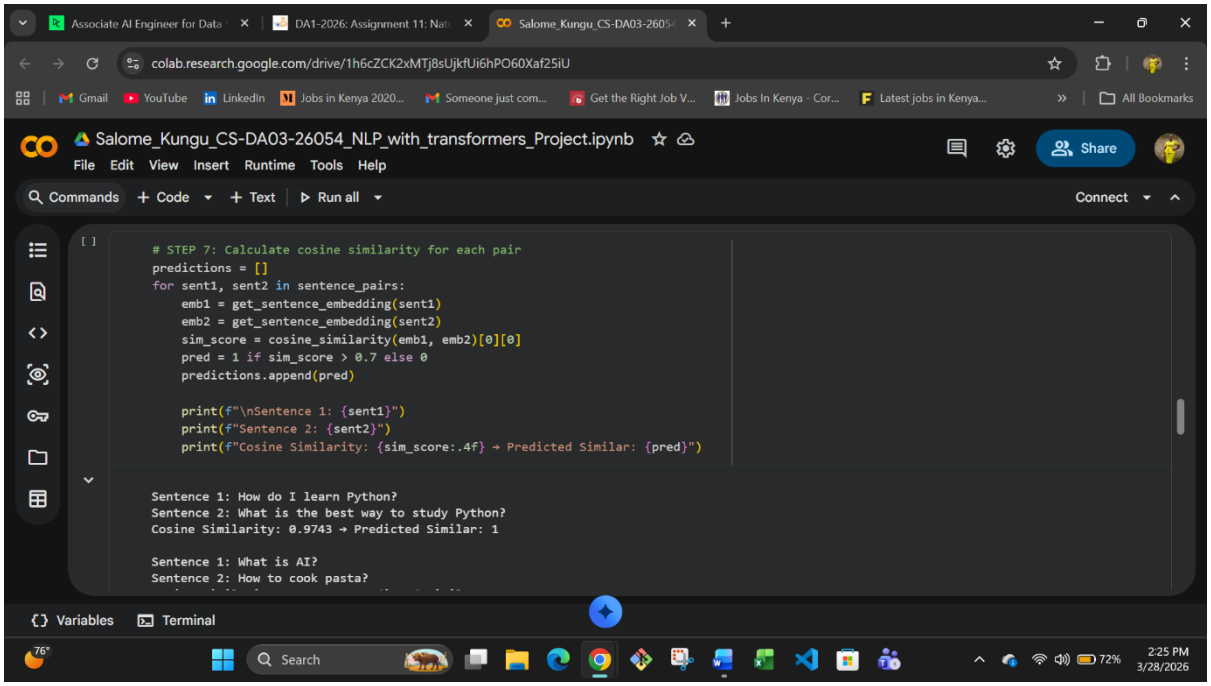
Task 4: Compute Cosine Similarity and Apply Similarity Threshold

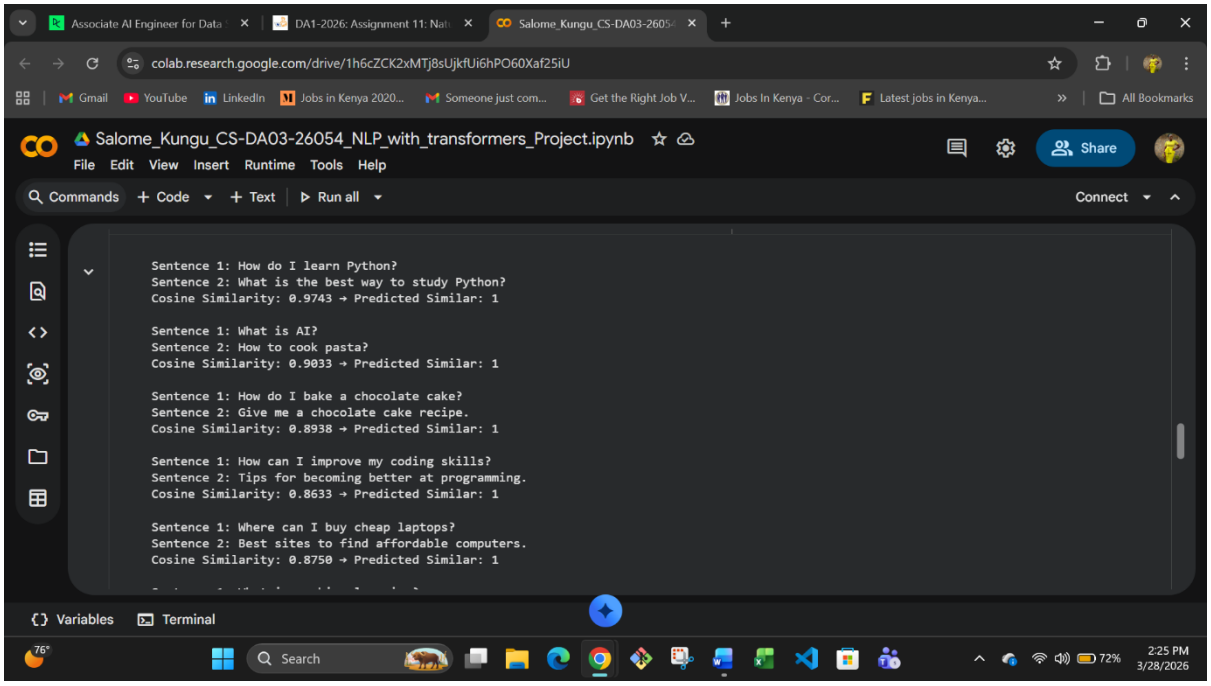
What

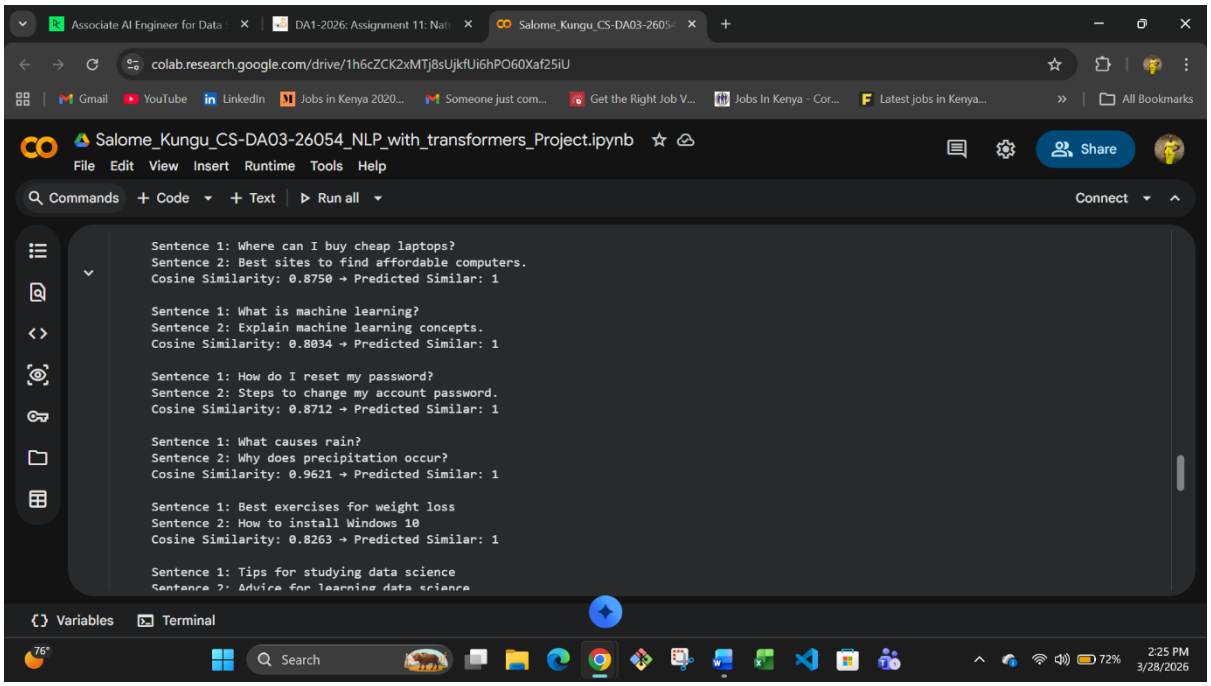
- Calculated cosine similarity between embeddings of sentence pairs.
- Applied a threshold of 0.7:
  - 0.7 → Similar (1)
  - ≤ 0.7 → Not Similar (0)

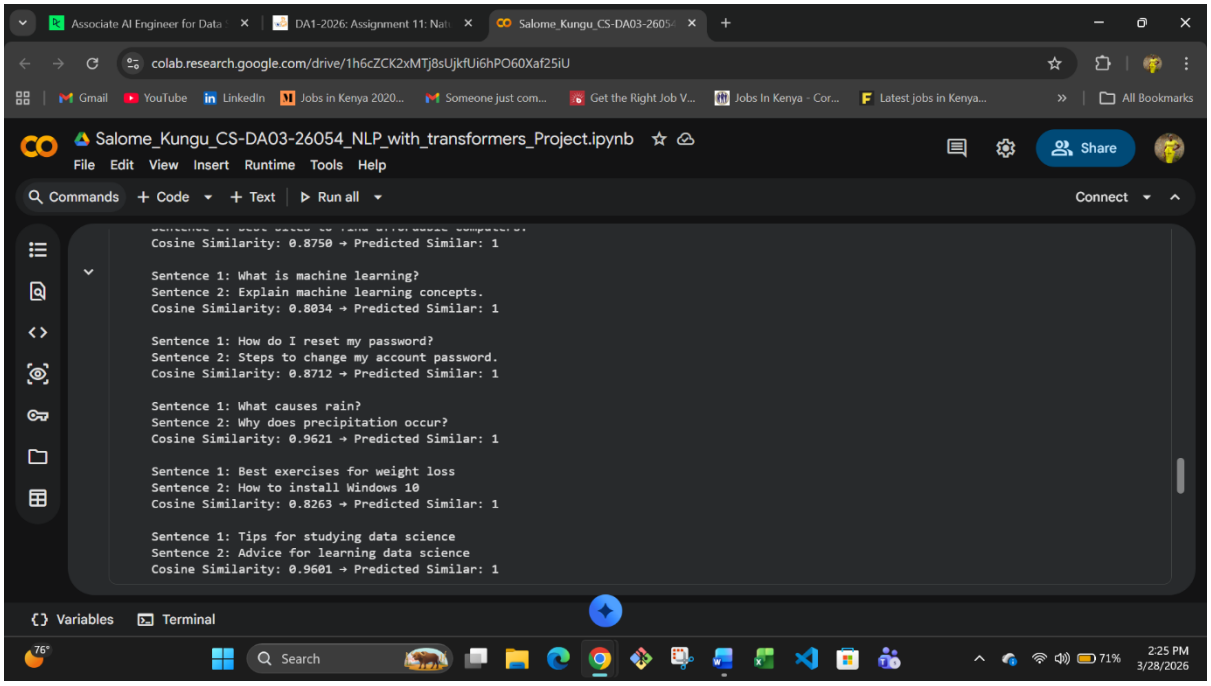
Why

- Cosine similarity measures how similar two vectors are in semantic space.
- This threshold helps classify sentence similarity.




Salome_Kungu_CS-DA03-26054_NLP_with_transformers_Project.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Connect
Sentence 1: How do I learn Python?
Sentence 2: What is the best way to study Python?
Cosine Similarity: 0.9743 → Predicted Similar: 1
Sentence 1: What is AI?
Sentence 2: How to cook pasta?
Cosine Similarity: 0.9033 → Predicted Similar: 1
Sentence 1: How do I bake a chocolate cake?
Sentence 2: Give me a chocolate cake recipe.
Cosine Similarity: 0.8938 → Predicted Similar: 1
Sentence 1: How can I improve my coding skills?
Sentence 2: Tips for becoming better at programming.
Cosine Similarity: 0.8633 → Predicted Similar: 1
Sentence 1: Where can I buy cheap laptops?
Sentence 2: Best sites to find affordable computers.
Cosine Similarity: 0.8750 → Predicted Similar: 1
Variables
Terminal


Salome_Kungu_CS-DA03-26054_NLP_with_transformers_Project.ipynb
File Edit View Insert Runtime Tools Help
Commands + Code + Text Run all
Share
Connect
Sentence 1: Where can I buy cheap laptops?
Sentence 2: Best sites to find affordable computers.
Cosine Similarity: 0.8750 → Predicted Similar: 1
Sentence 1: What is machine learning?
Sentence 2: Explain machine learning concepts.
Cosine Similarity: 0.8034 → Predicted Similar: 1
Sentence 1: How do I reset my password?
Sentence 2: Steps to change my account password.
Cosine Similarity: 0.8712 → Predicted Similar: 1
Sentence 1: What causes rain?
Sentence 2: Why does precipitation occur?
Cosine Similarity: 0.9621 → Predicted Similar: 1
Sentence 1: Best exercises for weight loss
Sentence 2: How to install Windows 10
Cosine Similarity: 0.8263 → Predicted Similar: 1
Sentence 1: Tips for studying data science
Variables
Terminal



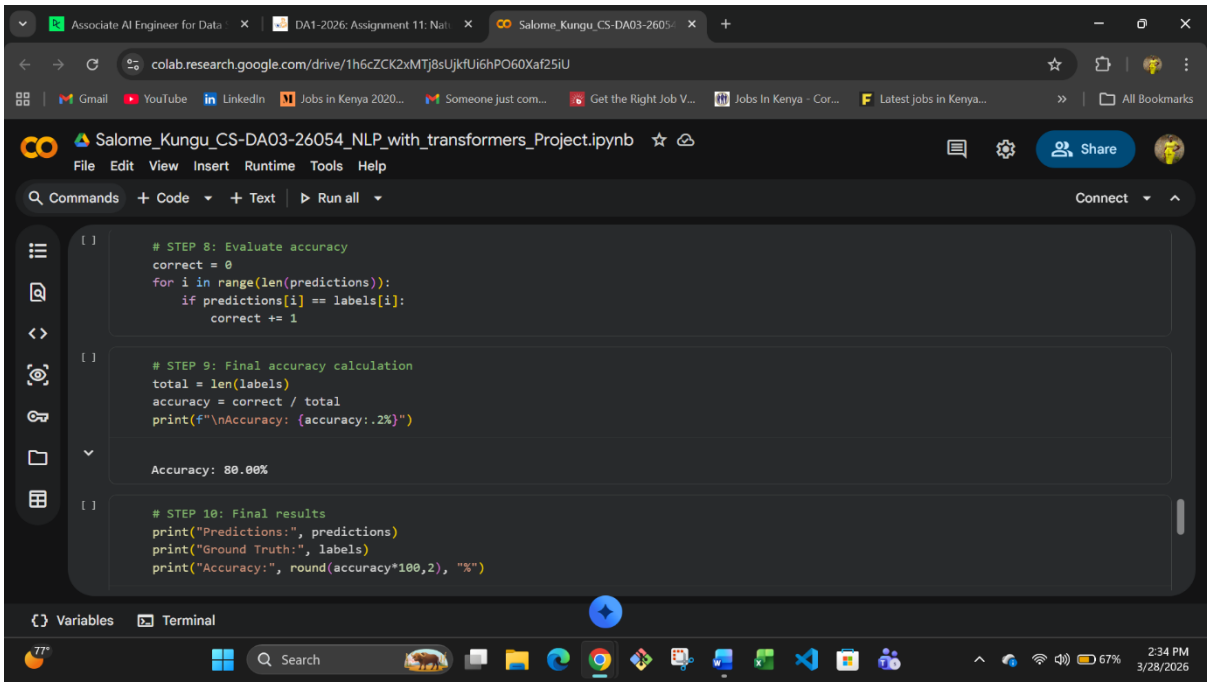
Task 5: Evaluate Model Accuracy

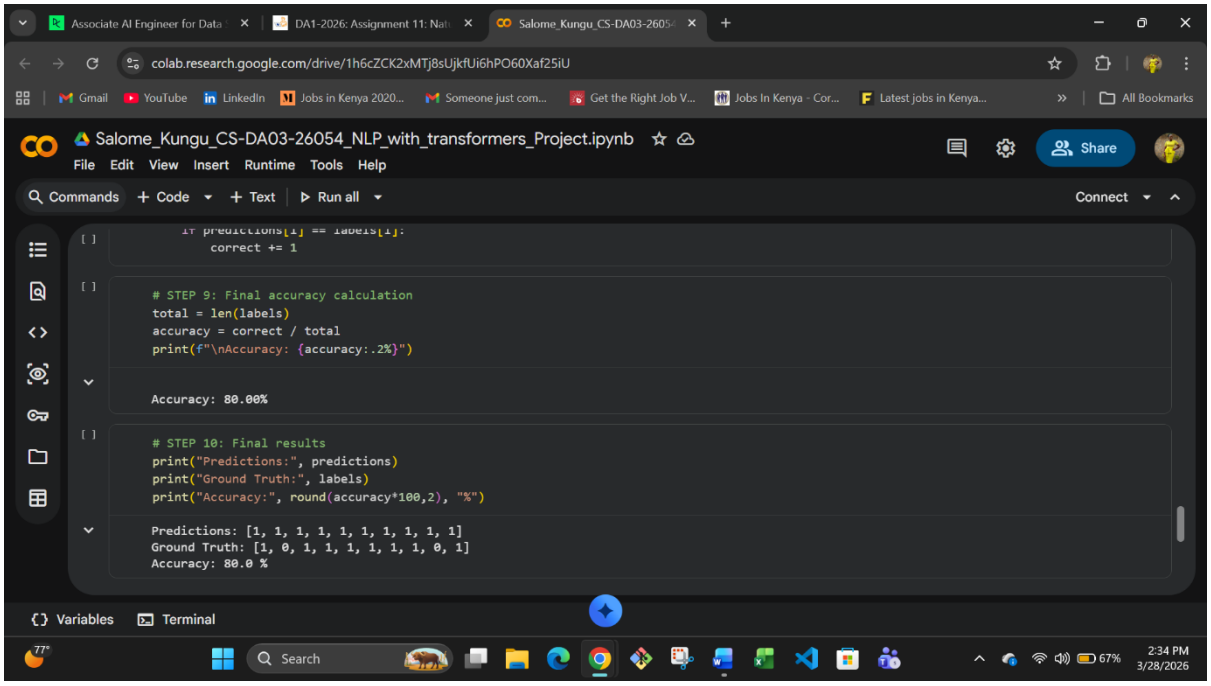
What

- Compared predictions with manually assigned ground truth labels.

Why

- Accuracy measures how correctly the similarity predictions were made.





Answers to the Extra Assignment Questions

1. How does BERT differ from traditional NLP approaches like Bag of Words or TF-IDF?

Traditional approaches like **Bag of Words** and **TF-IDF** represent text as fixed numerical vectors based on word frequency. These approaches ignore word order and context, meaning that words always have the same representation regardless of their usage.

BERT, however, generates **contextual embeddings**, meaning the representation of a word depends on the surrounding words in the sentence. This allows BERT to capture deeper semantic meaning and relationships between words.

2. What is the role of the encoder in the BERT model?

The encoder in BERT processes the input tokens using multiple **transformer layers with self-attention mechanisms**. These layers analyze relationships between all words in a sentence simultaneously.

In this assignment, the encoder transforms each sentence into a contextual embedding, which is then used to compute sentence similarity.

3. What are contextual embeddings?

Contextual embeddings are vector representations of words that depend on their surrounding context within a sentence.

For example, the word **"bank"** in:

- "I went to the **bank** to deposit money"

- "The boat reached the **bank** of the river"

will have **different embeddings** because the meaning differs based on context.

BERT generates these embeddings through its transformer encoder layers.

4. Why is the [CLS] token used for sentence similarity?

The [CLS] token is a special token added at the beginning of every input sequence in BERT. After passing through the transformer layers, the embedding of this token represents the **overall meaning of the entire sentence**.

Therefore, it is commonly used as a **sentence-level representation** for tasks like classification and similarity.

5. What is cosine similarity and why is it useful?

Cosine similarity measures the **angle between two vectors** in a high-dimensional space. It ranges from **-1 to 1**:

- **1 → identical vectors**
- **0 → unrelated**
- **-1 → opposite**

In NLP, cosine similarity is useful because it measures **semantic similarity between embeddings**, regardless of their magnitude.

Conclusion

This assignment demonstrated how transformer-based models such as BERT can be applied to Natural Language Processing tasks. By generating contextual embeddings for sentences, it was possible to measure semantic similarity using cosine similarity.

The experiment showed that BERT effectively captures contextual meaning and can distinguish between similar and unrelated sentences. Compared to traditional NLP methods like TF-IDF, BERT provides richer semantic representations due to its transformer architecture and contextual embedding capabilities.

Through this assignment, key concepts such as sentence encoding, contextual embeddings, cosine similarity, and transformer models were explored and implemented.